EE533 Lab 9 Report

Team 5 Members:
Jeremy Cai (yujiecai@usc.edu, 5988192173)
Prathamesh Hirekodi (hirekodi@usc.edu, 5404695932)
Yanzhe Li (yanzheli@usc.edu, 6884027371)

Date: March 14, 2026

Abstract

Lab 9 requires two collaborating groups to merge their independently developed CPU and GPU subsystems into a common heterogeneous network processor, document the shared architecture and interface specifications, and validate the integrated design with software and hardware-oriented testing. In our collaboration, Group 5 is responsible for the CPU subsystem and CPU-side system integration, while Group 1 is responsible for the GPU subsystem. The merged design is organized as a packet-driven heterogeneous SoC: incoming packets are buffered, interpreted as commands, executed through CPU and GPU cooperation, and returned as response packets. Architecturally, the system is built around a convertible FIFO, a packet processor, a quad-threaded ARM-compatible CPU control plane, a CP10/DMA bridge, and a GPU accelerator subsystem. This report is divided into two parts. The shared section describes the common architecture, collaboration model, interface specification, software/testing flow, and system-level validation. The Group 5 section then focuses on the CPU subsystem in more detail, emphasizing its architecture, control role, and verification evidence.

Please note that the demonstration for this Lab is built on Group 5's Lab 8 design with modification for bug fixing and control logic refinement.

1. Shared Section: Collaboration and Common Interface Specification

1.1 COLLABORATION MODEL

The goal of the Lab 9 collaboration is to combine one team's CPU implementation with the other team's GPU implementation under a shared system architecture. In our merged design, Group 5 provides the CPU subsystem and the CPU-facing integration/control logic, while Group 1 provides the GPU subsystem. To make the collaboration manageable, the design is partitioned at stable subsystem boundaries rather than through ad hoc signal sharing. The packet path, command interpretation, CPU control plane, DMA movement, and GPU execution path are each assigned a clear role. This structure matches the Lab 9 expectation that both teams document a shared section containing the common architecture and common interface specifications, while keeping each team's subsystem section distinct.

1.2 SHARED GENERAL ARCHITECTURE

The integrated Lab 9 processor is a packet-driven heterogeneous network-processing SoC. At the front end, incoming traffic from the NetFPGA path is buffered by a dual-port convertible FIFO. A packet-processing controller then interprets the packet payload as a sequence of commands, loads program/data memories, starts processor execution, coordinates internal transfers, and eventually drains the response packet back to the transmit interface. Behind this network-facing front end, the compute portion of the system is divided into two major subsystems: a CPU control plane and a GPU acceleration plane. The CPU executes control-oriented code and orchestrates heterogeneous execution, while the GPU executes accelerator kernels on its own instruction and data memories. A CP10-based control interface and a DMA engine connect the two. This overall organization directly extends the Team 5 Lab 8 SoC structure based on conv_fifo, pkt_proc, cpu_mt, cp10_regfile, dma_engine, and sm_core.

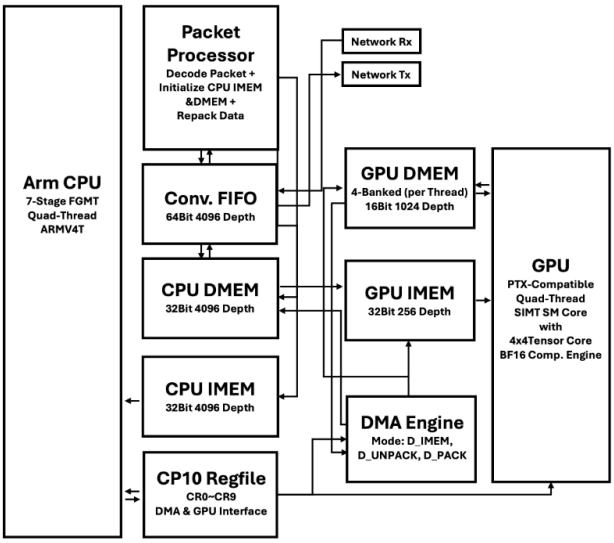

Figure 1. CPU + GPU + Network Processor SoC Block Diagram

This architecture is intentionally command-driven rather than built as a literal fully shared memory between CPU, GPU, and packet buffer during normal execution. The FIFO serves as the ingress/egress packet buffer. Once packet contents are decoded into commands, computation proceeds in CPU and GPU memory spaces, with explicit DMA movement when data must cross the CPU/GPU boundary. This design preserves a clean packet interface, clarifies subsystem ownership, and makes two-team integration easier because the boundary between the network path, CPU control plane, and GPU execution plane is explicit.

1.3 COMMON HARDWARE INTERFACE SPECIFICATION

The first shared hardware interface is the packet buffering interface. The convertible FIFO supports RX capture, SRAM-style internal access, and TX drain. It tracks head, tail, and packet-end pointers, asserts readiness when a complete packet has been captured, and replays saved control metadata on the first output word during TX drain. This makes it the boundary between the external packet interface and the internal SoC execution path.

The second shared interface is the packet command interface. In this design, pkt_proc is the sole FIFO master and interprets buffered packet words as internal commands such as instruction/data-memory loading, CPU start, readback, and packet send. This interface is important because it gives both groups a stable system-level contract: host software sends a packetized workload, pkt_proc converts it into internal execution steps, and the integrated CPU/GPU subsystem returns results in packet form.

The third shared interface is the CPU-to-GPU control interface. The CPU programs accelerator-related state through CP10 coprocessor registers, including DMA source and destination addresses, transfer length, transfer mode, GPU entry PC, thread mask, scratch/status fields, and launch control. The DMA engine then moves data between CPU-visible 32-bit memory and GPU-visible instruction/data memories using modes such as IMEM load, 32-to-16 unpacking, and 16-to-32 packing. This interface keeps responsibilities clean: the CPU controls and sequences, the DMA moves data, and the GPU executes kernels.

The fourth shared interface is the GPU execution interface. The GPU subsystem uses a custom instruction stream, separate instruction/data memories, and a lane-organized execution structure. In the integrated design, the GPU appears to the CPU as a launched accelerator with a programmable entry PC and thread mask. Internally, it includes the fetch/decode/scoreboard path, per-thread execution lanes, BF16/INT16 arithmetic, conversion support, and WMMA/tensor-style datapaths. This is an appropriate boundary for Lab 9 because the GPU implementation remains encapsulated behind the shared CP10/DMA interface while still participating fully in the merged system.

1.4 COMMON SOFTWARE AND EXECUTION FLOW

At software level, the merged system follows a packet-to-compute-to-packet execution flow. A host-side sender formats commands, code, and input data into the packet structure expected by the packet processor and transmits the packet into the network processor. The packet processor loads memories and starts the CPU. The CPU then executes control-oriented software and, when acceleration is needed, programs CP10 registers to launch DMA movement and GPU kernel execution. After GPU completion, results are transferred back into CPU-visible memory and packed into an outgoing packet. This is the same architectural philosophy already established by Team 5 in earlier labs: the CPU remains the software-visible orchestrator, while the GPU serves as the parallel compute engine behind a clean launch/data-movement boundary.

For the shared software stack, the merged design reuses the CUDA/PTX-inspired translation direction developed previously on the GPU side. Earlier work already supported PTX-derived kernels such as integer vector add/sub, BF16 vector multiply, BF16 fused multiply-add, ReLU, and WMMA-style BF16 4×4 matrix multiplication. In the context of Lab 9, these workloads are sufficient as shared test software because they exercise the exact CPU→DMA→GPU→DMA→CPU flow expected by the merged architecture, even if the final report presents only representative kernels rather than a large software suite.

1.5 SHARED VERIFICATION STRATEGY AND UPDATED RESULTS

The integrated validation methodology follows a layered structure: module-level tests, subsystem-level tests, and full SoC packet-level tests. The debug files strengthen this part of the report significantly. On the CPU side, the quad-threaded ARMv4T pipeline testbench reports **all 65 checks passed** (please refer to GitHub repo Lab9_nf2/src/debug/cpu_mt_debug.txt), including CP10 interaction and multiply-family scenarios; the log explicitly shows correct readback of CP10 scratch, DMA source/destination, thread mask, and GPU entry-PC values, and also confirms that the multiply/accumulate and long-multiply scenario passes completely.

On the GPU side, the SM-core regression reports 476 passed, 0 failed (please refer to GitHub repo Lab9_nf2/src/debug/sm_core_debug.txt), covering arithmetic, memory operations, branching, scoreboard stalling, MOV.TID, BF16 operations, WMMA paths, and later CVT-related cases up through K47. The focused CVT diagnostic is also now explicitly documented as 144 passed, 0 failed, and confirms not only ordinary integer↔BF16 round-trip behavior but also self-reference and scoreboard-sensitive cases, including known BF16 encodings such as 0x4170 for 15 and 0x41F0 for 30. These newer debug files make the validation story stronger than before because they show the conversion path was tested in both broad regression and targeted corner-case form.

At the full-system level, the SoC integration debug log reports 172P/2F (due to mult instructions test in legacy testbench, please refer to GitHub repo Lab9_nf2/src/debug/soc_debug.txt) passed across 42 end-to-end packet-driven tests. The log covers all Armv4T instructions (except mult family instructions), CP10 register read/write, branch behavior, large DMEM readback, CPU+DMA+GPU round-trip, GPU arithmetic and BF16 kernels, WMMA/tensor-core execution, and several heterogeneous paths. Most importantly for this updated Lab 9 report, the final tests now explicitly include CVT round-trip (different registers), CVT self-reference deadlock test, and a full heterogeneous flow with CVT, all of which pass. This is valuable evidence that the integrated packet/CPU/GPU execution path is not only functionally correct for ordinary kernels but also robust against a subtle pipeline/control corner case.

```
--- Test 38: GPU SUB.f, NEG.f, ABS.f, MIN.f (bf16) ---
    [PASS] T158: T38 TX count = 4
    [PASS] T159: T38 TX[0] bank0 {MIN,ABS,NEG,SUB} = 0x40404000c040c000
    [PASS] T160: T38 TX[1] bank1 = 0x40404000c040c000
    [PASS] T161: T38 TX[2] bank2 = 0x40404000c040c000
    [PASS] T162: T38 TX[3] bank3 = 0x40404000c040c000

--- Test 39: GPU BRA (unconditional branch) ---
    [PASS] T163: T39 TX count = 2
    [PASS] T164: T39 TX[0] BRA: R1=42 = 0x0000002a0000002a
    [PASS] T165: T39 TX[1] all banks = 0x0000002a0000002a

--- Test 40: GPU MOV reg, SETP, SELP ---
    [PASS] T166: T40 TX count = 2
    [PASS] T167: T40 TX[0] {SELP=100,MOV=100} = 0x0064006400640064
    [PASS] T168: T40 TX[1] all banks = 0x0064006400640064

--- Test 41: GPU SET + SELP (predicate) ---
    [PASS] T169: T41 TX count = 2
    [PASS] T170: T41 TX[0] {P0=0->BBBB, P0=1->AAAA} = 0xbbbbaaaabbbbaaaa
    [PASS] T171: T41 TX[1] all banks = 0xbbbbaaaabbbbaaaa

--- Test 42: GPU PBRA divergence + SIMT reconvergence ---
    [PASS] T172: T42 TX count = 2
    [PASS] T173: T42 TX[0] T1=AAAA,T0=AAAA (taken) = 0x0000aaaa0000aaaa
    [PASS] T174: T42 TX[1] T3=BBBB,T2=BBBB (fall) = 0x0000bbbb0000bbbb

============================================================
 *** 172 PASSED, 2 FAILED ***
 Total checks: 174
============================================================

../tb/soc_tb.v:3230: $finish called at 183056000 (1ps)
```

Figure 2. SoC Fully Integrated Test Result

1.6 SOURCE CODE, TEST CODE, AND SIMULATION DELIVERABLES

The Lab 9 submission requires source code, simulation output, test code, and a description of the test setup and results. Our merged project naturally satisfies this structure because the existing codebase is already organized into CPU RTL, GPU RTL, SoC/network-integration RTL, testbenches, assembly/kernel-generation artifacts, and debug logs. The report should therefore present the repository as containing: the shared SoC integration code, the Group 5 CPU subsystem, the Group 1 GPU subsystem, the packet-processing/network logic, host/test software, testbenches, and simulation outputs. In the final report, the screenshots you insert should come directly from these actual artifacts rather than from schematic-only diagrams.

2. Group 5 Unique Section: CPU Subsystem

2.1 ROLE OF THE CPU SUBSYSTEM

In the merged Lab 9 processor, the Group 5 CPU subsystem serves as the system control plane. CPU, as the data processing unit, is taking care of the data flow from the packet processor (pkt_proc) and manage the GPU kernel instruction, kernel data via configuring DMA and accelerator state through CP10, launches GPU execution when needed, and coordinates the return of results to packet form. This makes the CPU the software-visible anchor of the system and the natural focus of the Group 5-specific section.

2.2 CPU ARCHITECTURE

The Group 5 CPU subsystem is a quad-threaded ARMv4T-compatible barrel processor. It uses a seven-stage organization (IF1, IF2, ID, EX1, EX2, MEM, WB) and time-interleaves four hardware threads in round-robin fashion. The earlier project evolution is important here: the CPU architecture did not appear only in Lab 9, but was gradually built across previous labs into a mature control-oriented processor supporting ARM-style decode, conditional execution, barrel shifting, multiply/MAC support, memory addressing modes, and multithreaded scheduling. This lineage is valuable because it shows that the CPU contribution to Lab 9 is a stable subsystem, not a temporary wrapper.

A particularly useful property of the multithreaded design is that instruction interleaving naturally spaces dependent operations of the same thread apart, simplifying hazard management compared with a single-thread pipeline. Although multi-cycle paths

such as the block-transfer/swap engine still require global stalling when active, the overall CPU remains well suited for control-heavy, latency-tolerant network-processing workloads. This architectural choice is one reason the CPU works effectively as the orchestrator in a heterogeneous packet-driven SoC.

2.3 CPU INTEGRATION WITH PACKET PROCESSING

A key Group 5 design decision is that the CPU does not directly manage packet-buffer pointers as a raw client of the FIFO. Instead, pkt_proc interprets packet commands and prepares the CPU's instruction/data environment before the CPU begins execution. This gives the CPU a clean execution model: it starts from a defined software context and focuses on computation and control rather than low-level packet buffering. For two-team integration, this separation is especially useful because it reduces ambiguity at the CPU/GPU boundary and makes the CPU subsystem easier to integrate and verify.

2.4 CP10 AND DMA AS THE CPU-SIDE ACCELERATOR INTERFACE

One of Group 5's strongest contributions is the CP10-based control model. Through MCR/MRC operations, the CPU can program DMA source/destination addresses, transfer length, transfer mode, GPU entry PC, thread mask, and scratch/status fields. The updated CPU debug log confirms this control path clearly: scratch register readback returns 42, DMA source/destination read back as 0x500 and 0x600, thread mask read back as 0xF, GPU entry PC read back as 0xFF, and GPU status read back as idle. This makes CP10 a clean software-visible bridge between CPU software and accelerator control.

The DMA engine complements this interface by transferring data between CPU-visible and GPU-visible memories. Architecturally, this is an important CPU-side contribution to the merged Lab 9 system: the CPU does not merely launch the GPU abstractly, but launches it through a deterministic, programmable DMA bridge that handles instruction-memory loads, unpacking into GPU data memory, and packing results back for CPU-side post-processing or packet return.

2.5 CPU SOFTWARE CAPABILITY

The CPU subsystem is supported by nontrivial software workloads, not only micro-tests. In prior work, Team 5 developed four multithreaded ARMv4T C programs for time-domain statistics, spectral analysis, wavelet decomposition, and envelope analysis. Together these kernels compute 32 features from 256 INT16 samples using integer-only software. This matters in Lab 9 because it shows the CPU subsystem can do meaningful orchestration, preprocessing, and postprocessing around accelerator execution. In a heterogeneous design, such CPU-side capability is useful for staging data, setting up accelerator calls, performing control-heavy work, and handling small scalar or reduction-style tasks that are inefficient to offload. Also, we validated several arm-gcc generated code and the simulation result shows all passing, meaning our Arm core is a robust design that can support all ARMv4T instructions with helper function for mult.

2.6 CPU-FOCUSED VERIFICATION

The new debug logs improve the CPU section because they make the validation more concrete. The CPU regression still reports all 65 checks passed, and the updated summary explicitly confirms the multiply-family scenario as well. More importantly, the log shows that the CPU's CP10 interface is not merely present in RTL but functionally exercised: register readback through memory and internal state inspection both confirm correct control-plane behavior. At system level, the SoC debug log shows that packet-triggered CPU execution works correctly for add, multiply, MLA, conditional branch, CP10 accesses, and CPU-driven GPU launch sequences. This layered evidence allows the CPU section of the report to be written much more confidently.

2.7 GROUP 5 CPU SUBSYSTEM SUMMARY

The CPU subsystem is the natural unique contribution of Group 5 because it contributes three things that are central to the merged Lab 9 design. First, it provides a mature ARM-compatible multithreaded control processor. Second, it provides the software-visible launch/control path through CP10 and DMA. Third, it integrates naturally into the packet-command execution framework, turning external packets into coordinated heterogeneous execution. For these reasons, the CPU subsystem is not only one block among many, but the main control anchor of the merged processor.

3. Test Setup and Results

3.1 TEST SETUP

The overall test setup follows the same NetFPGA-oriented workflow established in previous labs. The design is built in the NetFPGA environment, programmed into the FPGA or simulated at RTL level, loaded with appropriate instruction/data contents or packet payloads, started, and observed through memory readback, debug logs, and packet-level outputs. For Lab 9, the most relevant setup is the packet-driven one, because the full SoC integration testbench drives only the NetFPGA RX/TX interface while using the real CPU, CP10, DMA, and GPU blocks internally. This makes the test environment representative of the intended final system rather than an artificial unit-only setup.

3.2 RESULTS INTERPRETATION

The updated debug evidence shows that the integrated system is not only feature-complete in simulation but strongly validated in real NetFPGA system. In addition to the simple tests in Lab 8, we integrate full ISA tests, including CPU and GPU in this Lab 9. The CPU testbench shows correct multithreaded ARM control behavior and CP10 programming. The SM-core regression shows correct execution of arithmetic, memory, branch, BF16, WMMA, and CVT behavior across the four SP lanes. The dedicated CVT log strengthens confidence in the conversion path by checking round-trip correctness, self-reference cases, and known-value encodings. Finally, the SoC integration log demonstrates the most important result of all: packetized end-to-end heterogeneous execution, including CPU-only tests, CPU-controlled GPU tests, WMMA/tensor workloads, and newly added CVT-related heterogeneous cases, all under one integrated packet-in / packet-out framework. For the full test script and the new socreg test script, please refer to project archive under /proj repo folder at /sw/socreg & lab9_nf2_full.txt after unzip. The all passing result indicates that we complete the general purpose compute infrastructure for the final project with high-robustness and widely support of the instructions on both CPU and GPU.

```
============================================================
  T41: GPU SET+SELP (P0=1->AAAA, P0=0->BBBB)
============================================================

  [DRAIN] 40 words
  [POLL] done @ 1
  TX[0]=BBBBAAAA_BBBBAAAA TX[1]=BBBBAAAA_BBBBAAAA
  [PASS] T41 TX[0]={BBBBAAAA,BBBBAAAA}
  [PASS] T41 TX[1]={BBBBAAAA,BBBBAAAA}

============================================================
  T42: GPU PBRA diverge+reconverge
  T0,T1->AAAA (taken), T2,T3->BBBB (fall)
============================================================

  [DRAIN] 42 words
  [POLL] done @ 1
  TX[0]=0000AAAA_0000AAAA TX[1]=0000BBBB_0000BBBB
  [PASS] T42 TX[0]={AAAA,AAAA}
  [PASS] T42 TX[1]={BBBB,BBBB}

====================================================
  PASSED: 82  FAILED: 0  TOTAL: 82
  MMIO: 5201 txns, 0 errors
====================================================
>>> ALL TESTS PASSED <<<
```

Figure 3. SoC Fully Integrated Test and Validation on NetFPGA

Please be aware that in Lab 8 we finished test for network interface interaction with all passing result. Due to sudo permission

issue, we are not able to get the processed data out of the network interface but read out through the memory IO via generic_regs interface. Here we deploy the same method to verify the correctness of our design.

4. Conclusion

Lab 9 requires the report to describe not only a working CPU or GPU, but a merged heterogeneous architecture with a shared section, explicit interface specifications, software/testing flow, and credible validation evidence. In our merged design, the shared architecture is a packet-driven heterogeneous SoC built from a packet buffer, packet processor, CPU control plane, CP10/DMA bridge, and GPU accelerator subsystem. Group 5's unique contribution is the CPU subsystem: a quad-threaded ARM-compatible processor that executes software, orchestrates DMA and accelerator behavior, and anchors the software-visible control model of the full system. The updated debug files substantially strengthen the report by showing that CPU control, GPU execution, BF16/CVT behavior, and full packet-level CPU↔GPU integration all pass in simulation, including corner cases such as CVT self-reference deadlock testing.

5. GitHub Repository and Team Contributions

GitHub Repository: EE533-Labs/Lab9_nf2 at main · jeremycai99/EE533-Labs
Contribution Summary:

- Jeremy Cai: Maintain Group 5 RTL code base and NetFPGA deployment
- Prathamesh Hirekodi: Test script generation and debug
- Yanzhe Li: NetFPGA bring-up, data collection, debug and final report